

Mapping Corporate Ethics: Clustering UK Companies' Values Using NLP

Topic Proposal

Johannes Van Cauwenberghe

2024-08-21

Table of contents

1. Introduction	1
2. Data Extraction	2
3. Technology and Limitations	2
4. Hypotheses and Research Questions	4
5. Planned Analysis	4
6. Data Structure and Summary of Exploratory Analysis	5
6.1 The Structure of the Data	5
6.2 CC-Index Files: The Manifest	6
6.3 CC-Index Files: The Structure	6
7. Exploratory Analysis	7
8. Summary and Conclusions	8
8.1 WARC and WET Files: An Unexpected Challenge	8
8.2 Conclusion	9
9. References	10

1. Introduction

The challenge of comparing corporate performance in areas such as social responsibility, environmental commitments, and governance is well-documented. Environmental, Social, and Governance (ESG) data providers, including Clarity.ai, Briink, Sustainalytics, Datamaran, RepRisk, and Factset (formerly Truvalue Labs), offer various methodologies and data sources designed to meet the growing demand for sustainable investment strategies. RepRisk, for example, is said to aggregate data from over 80,000 sources daily, including media, social media, and other public platforms, to deliver analytics that evaluate the impact of ESG issues

on company compliance, financial performance, and reputational risks ([Bose and Springsteel, 2017](#)).

This project proposes leveraging the Common Crawl dataset to map and cluster the corporate values of UK companies by analysing large volumes of web pages. Specifically, the project will focus on identifying and clustering corporate values and mission statements to understand the landscape of corporate social responsibility (CSR) in the UK.

The project will utilise Hadoop HDFS and Apache Spark to process the Common Crawl's extensive dataset. The primary research question is whether there are discernible patterns in how companies articulate their values and what these patterns reveal about the corporate landscape in the UK.

To address these questions, the project will develop a data processing pipeline that filters, reshapes, and refines a subset of the Common Crawl dataset. This pipeline will include a bespoke classifier for identifying corporate values, followed by clustering and topic modelling algorithms to extract insights. The ultimate goal is to develop a generalisable pipeline that can be scaled and deployed on cloud platforms such as AWS Elastic MapReduce (EMR) or Google Dataproc.

2. Data Extraction

Data acquisition is the foundational step in this project, providing the necessary raw material for subsequent analysis. The latest Common Crawl dataset, comprising 327.4 TiB of uncompressed data stored on Amazon S3, presents a unique opportunity to explore corporate communications at scale. Common Crawl provides a variety of use cases and examples on its website, and its PySpark [GitHub library](#) is rich in use cases for both Common Crawl and PySpark.

This project will process corporate websites sequentially, beginning with an exploratory analysis to understand the dataset's structure and content. In the proposal below, we will start by contextualising our approach within the technological resources and constraints. We will then formulate research questions that will guide our effort. Next, we will break down the analytical phase of the project and give a preliminary overview of the structure of the data in an exploratory data analysis. This is the main section of this project proposal. There, we will summarise key findings and address key challenges, which leads into the conclusion section where we will set out how we addressed the challenges and present our results.

3. Technology and Limitations

As per coursework specifications, this project will utilise Hadoop HDFS for data storage and Apache Spark for data analysis, leveraging Spark's MLlib for machine learning tasks. Given

its ability to handle large-scale data processing efficiently, PySpark is particularly suitable for this project.

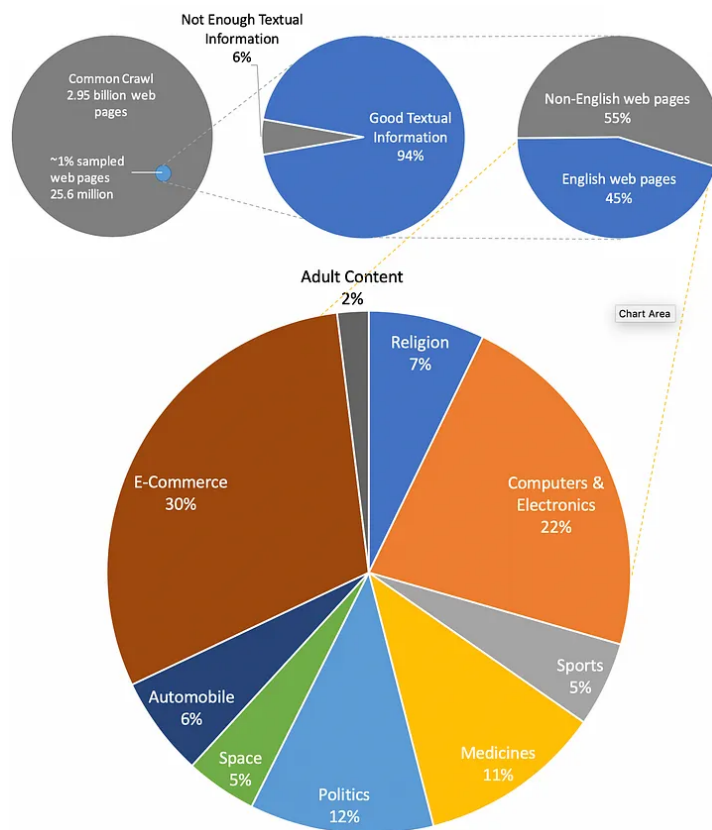


Figure 1: Categorising the World Wide Web. Source: Pavagadhi (2019).

Pavagadhi (2019) demonstrates, using PySpark technology, that we can process 25.6 million web pages, approximately 1% of the Common Crawl corpus, in about 1.5 hours (on 100 EC2 instances on AWS). We note the resource constraints on the Lena cluster are as follows:

- **Cluster:** The available cluster consists of 20 nodes with 4 vcores each, and a maximum of 160 GiB of memory, of which 45 GiB is typically in use. Given 2-4 partitions per CPU core, we will aim to split large workloads over 40-80 partitions, such as when processing the index files.
- **Hadoop:** The HDFS has 36 TB of available space out of 48 TB total. This does not pose any notable constraints. The index and WET files (details follow below) should take no more than a couple of Gigabytes.
- **Networking:** While direct downloads from Amazon S3 to worker nodes (or HDFS) are not feasible the project will download onto the local node on the Lena server and then move the data into HDFS.

4. Hypotheses and Research Questions

This project serves multiple objectives, with a primary focus on establishing a proof of concept for clustering corporate values using the Common Crawl dataset. The specific objectives are:

- Identify and distinguish corporate social responsibility (CSR) pages using Common Crawl Index files.
- Batch-process large volumes of web pages from the Common Crawl and funnel them into our classifier based on index files and metadata.
- Identify distinct clusters of companies with similar descriptions of their values and corporate identities, allowing for meaningful comparisons of corporate philosophies.

5. Planned Analysis

The project will follow a structured approach:

1. Data Extraction:

The project will use PySpark in a Jupyter Notebook to iteratively and interactively filter and extract relevant URLs and download the corresponding WET files. This will be a challenge, as this requires navigating the already substantial index files and handling unfamiliar file formats with each file containing tens of thousands of rows. We walk through the key learnings below in the [Data Structure and Summary of Exploratory Analysis](#).

2. Data Preprocessing:

Next is the preprocessing of the text data. This will include the use of regular expressions to further clean the data, as well as the usual pipeline of tokenisation, stopword removal, and TF-IDF. Given the unstructured nature of web data, preprocessing is critical for accurate classification and clustering.

3. Text Classification:

Once we have obtained a useful subset of clean data, we will use a classifier to categorise the text into predefined categories. Taking inspiration from Pavagadhi (2019), we will use a simple text classifier, trained on `fetch_20newsgroups`. The goal is to transfer the classification learned from the `fetch_20newsgroups` dataset to categorise the webpages into similar predefined categories (e.g. `sci.med`, `alt.atheism`). For this purpose we will use Naive Bayes. Naive Bayes is a parametric algorithm known for requiring little training and particularly suited for text classification.

4. Clustering:

We can now apply the clustering algorithms to group companies based on their stated values. This step will validate our approach. The project will include the K-means as well as Bisecting K-means. We also include a topic modelling implementation using

Latent Dirichlet allocation (LDA). In contrast to k-means clustering, LDA assigns each webpage to multiple topics.

5. Evaluation:

Finally, we will assess the performance of the classification models, the clustering algorithms, and LDA. This last step will feed into the Summary and Conclusions section below.

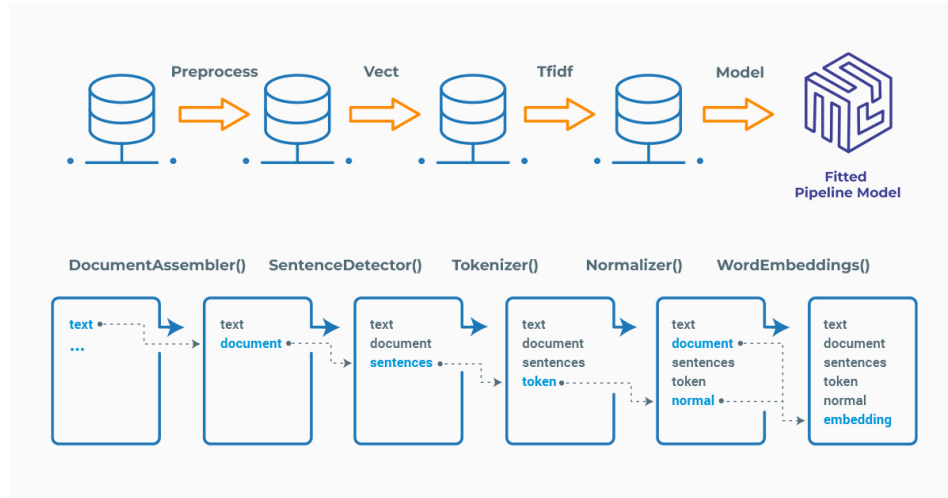


Figure 2: The NLP preprocessing pipeline used in this project. Source: [Pirge \(2023\)](#)

6. Data Structure and Summary of Exploratory Analysis

Our initial exploratory analysis of the Common Crawl dataset reveals an immense scale and complexity. This section will detail the structure of the dataset, the size of its components, and the results of the exploratory analysis on the filtered subset.

6.1 The Structure of the Data

The dataset structure is organised into several key components. It is navigated via manifest files.¹ Each manifest file, identified by the `.paths` extension, is a gzipped list of file paths, typically occupying only a few kilobytes.

Each of these file paths references a tab-separated index file. These index files, in turn, reference the raw HTML outputs stored in WARC files. This is raw web content.

¹The main interface of the manifest can be viewed on data.commoncrawl.org.

6.2 CC-Index Files: The Manifest

The `cc-index.paths.gz` manifest file consists of 302 URL paths to cc-index files, each serving as an index for a portion of the Common Crawl dataset (as seen in Schaffhauser (2022)). These files follow a structured naming convention:

```
['cc-index/collections/CC-MAIN-2024-30/indexes/cdx-00001.gz',  
 'cc-index/collections/CC-MAIN-2024-30/indexes/cdx-00002.gz',  
 ...  
 'cc-index/collections/CC-MAIN-2024-30/indexes/cdx-00299.gz',  
 'cc-index/collections/CC-MAIN-2024-30/indexes/cluster.idx',  
 'cc-index/collections/CC-MAIN-2024-30/metadata.yaml']
```

Each cc-index file is approximately 750 MB in size when compressed, but expands to around 5.58 GB, due to a compression ratio of approximately 7.74. These indices will be critical for navigating the vast amount of data within the Common Crawl.

One of the cc-index files, `cluster.idx` (shown on the second to last line above), contains the first line of each of the 302 cc-index files. This is important to note because it provides a starting point for our preprocessing pipeline. As the index collection is organised by SURT (Sort URL Tokenisation) keys, preselecting specific cc-index files enables an effective first selection. For example, it is noted that *seven idx files* begin with the prefix `uk`. This brings the compressed dataset to 5.2 GiB, which is feasible.

6.3 CC-Index Files: The Structure

The cc-index files provide valuable metadata, including top-level domain names, page counts, and URL segments. This metadata allows for filtering based on criteria such as language, status codes, and MIME types:

```
['_corrupt_record',  
 'charset',  
 'digest',  
 'err',  
 'extpvdrr',  
 'filename',  
 'isext',  
 'languages',  
 'length',  
 'mime',  
 'mime-detected',  
 'offset',
```

```
'redirect',
'status',
'truncated',
'url']
```

For the purposes of this project, we will retain only *successful* scrapes *in English* encoded in *utf-8*. This filtering allows us to focus on text data that is most relevant to the research objectives.

7. Exploratory Analysis

After applying the preselection criteria, we can now summarise the results based on the URLs retrieved from the index. By processing the seven `idx` files containing the `uk` SURT key, we collected a total of 55,506,179 records. Of these, 45,127,115 URLs ended with the `.co.uk` domain.² Within this subset, we identified 741,564 unique domains, of which 190,182 are single-page domains, which we can discard. We focused on 423,813 larger domains—defined as domains with more than five pages—which represent a total of 33,911,119 pages. These larger domains were flagged for further analysis.

We then selected a sample of 7,972 URLs based on the presence of specific CSR-related terms such as ‘esg’, ‘strategy’, ‘csr’, ‘vision’, ‘purpose’, ‘about’, ‘mission’, and ‘values’. The distribution of these terms is as follows:

Table 1: Extracted URL Segments by Count

CSR Segment	Count
esg	2868
strategy	1744
csr	1246
vision	808
purpose	492
about	345
mission	235
values	234

²Note that we only see the first line in `cluster.idx`.

8. Summary and Conclusions

8.1 WARC and WET Files: An Unexpected Challenge

In processing these URLs, we encountered challenges related to the formats of the scraped web pages, specifically the WARC and WET file types. WARC (Web ARChive) files store the raw crawl data, while WET files are smaller (~85 MB) and only store the extracted plaintext from the WARC files.

For our analysis, we had planned to selectively retrieve the WARC files based on their metadata. Each line in the index files includes the URL, offset, and length of the webpage as a `bytestring` in WARC format. This allows us to precisely target specific sections of the WARC files, as shown in the documentation at commoncrawl.org:

```
requests.get(warc_url, headers={'Range': f'bytes={offset}-{offset+length-1}'})
```

However, WET files, which are more efficient for text analysis, cannot be retrieved in such a targeted manner because they lack the same byte-range indexing. Since each WET file holds approximately 20,000 web pages, the number of paths associated with our sample of nearly 8,000 URLs became too large for this approach to be feasible.

Although WET files are useful for random samples of web content (see [Pavagadhi, 2019](#); [Suen, 2018](#)), this limitation required us to explore alternative strategies for optimising data extraction and processing.

- **Processing WARC Files:** One option would be to process WARC files instead of WET files. This approach would require using the BeautifulSoup library for HTML parsing, which is not readily available on the cluster. This could be solved by packaging and distributing a conda environment via Spark's `--archives` option, as outlined [here](#). However, this would involve thousands of API requests, which introduces additional complexities.
- **Scraping Directly from URLs:** Another approach would be to bypass the Common Crawl dataset altogether and scrape the target URLs directly. This may prove more efficient but moves away from the original scope of the project.
- **Random Sampling of WET Files:** We could pivot the project to focus on a random sample, as seen in prior studies (e.g., [Pavagadhi, 2019](#); [Suen, 2018](#)), but this would require a different setup, as WET files contain web pages in multiple languages, necessitating additional language detection and filtering.

The final approach, which was adopted in this project, involved prioritising the WET files containing the highest number of target documents. This was done by sorting the WET filenames based on their frequency of occurrence in the dataset. By processing only the most frequent files (approximately 300), we were able to optimise resource usage, minimise costs, and reduce network overhead. This strategy allowed the project to proceed within the constraints

of a large-scale data environment while still yielding meaningful insights for the clustering and topic modelling analysis.

8.2 Conclusion

This project aimed to use the Common Crawl dataset to map and cluster the corporate values of UK companies by analysing large volumes of web pages. While the initial objective led to significant data exploration and development of a scalable data pipeline, the project encountered several challenges that impacted the final outcomes.

The methodology began by identifying and filtering relevant URLs from Common Crawl's index files. Using the `uk` SURT key, we identified over 45 million `.co.uk` URLs, which were narrowed down to approximately 8,000 based on the presence of key CSR-related terms. These URLs formed the basis of the data that we would analyse for corporate values.

To process the data, we relied on a pipeline involving Hadoop HDFS and Apache Spark for large-scale data management. Preprocessing was a crucial step, given the noisy and unstructured nature of web data. This involved applying regular expressions and standard text cleaning techniques such as tokenisation, stopwords removal, and TF-IDF. Despite these efforts, a significant amount of noise remained in the data, particularly due to the presence of text elements like Javascript and CSS. Particularly the relatively rare `TailwindCSS` tokens appeared intractable.

For text classification, we implemented a Naive Bayes classifier, trained on the `fetch_20newsgroups` dataset, to categorise the web pages into CSR-related categories. Naive Bayes was selected for its efficiency in handling text classification tasks and its low computational requirements. However, the classifier's performance was hindered by the noise present in the data. Despite applying regular expressions to clean the documents, a significant amount of residual noise persisted, leading to suboptimal results. We experimented with feature selection techniques, such as Chi-Square (χ^2), and tested alternative algorithms like logistic regression and random forests. Unfortunately, these experiments did not yield improved results, and as such, we decided not to include them in our final analysis.

When we moved to clustering, similar challenges persisted. The high level of noise and the relatively small number of clean documents (approximately 269) resulted in incoherent clusters. The silhouette score of -0.019 reflected poor cluster separation. The elbow method also failed to provide clear insights, likely due to the inconsistencies in the data. LDA topic modelling proved equally challenging to interpret. The model produced a very low log likelihood ($-6e5$), indicating a poor fit (higher values are better), while the perplexity stabilised around 7, which is considerably high (lower values indicate better predictive performance), suggesting the model struggled to generalise the data effectively.

Several improvements could be made to address these challenges in future iterations. One potential approach would be to manually label a subset of high-quality web pages, which could

then be used to train a more targeted binary classifier. This would help better distinguish between relevant and irrelevant content, improving the overall classification accuracy. Alternatively, regular expressions could be further refined to automate some of this labelling process. This would allow the classifier to filter out noise more effectively, leading to better clustering and topic modelling results.

Despite these challenges, the metadata contained in the Common Crawl index provided useful insights into UK domains. For instance, it was notable that nearly 200,000 domains consisted of only a single page, while larger domains (with more than five pages) averaged about 80 pages. The use of regular expressions on these domains to filter common URL segments yielded some promising results. However, in future projects, it may be more efficient to scrape the selected URLs directly, rather than rely on the Common Crawl’s WET files, as their unstructured nature made incremental filtering and extraction intractable.

In conclusion, while the project faced challenges, it offered valuable insights into the practical limitations of working with large-scale web data. It also provided a deeper understanding of PySpark and Common Crawl’s capabilities and constraints. Moving forward, a focus would be to refine the data pipeline and explore alternative classification methods to handle noisy and unstructured content more effectively.

The Common Crawl dataset remains a powerful resource for large-scale analysis. With the right approach, it has the potential to offer valuable insights into corporate and consumer behaviour, including in the context of CSR and corporate identity. As new crawls are released regularly, the dataset continues to provide a constantly evolving snapshot of the web, offering new opportunities for research and business intelligence.

9. References

- Bose, S., Springsteel, A., 2017. The value and current limitations of ESG data for the security selector. *Journal of Environmental Investing* 8.
- Pavagadhi, J., 2019. Categorizing the world wide web [WWW Document]. URL <https://towardsdatascience.com/categorizing-world-wide-web-c130abd9b717>
- Pirge, G., 2023. How to detect languages with python: A comprehensive guide [WWW Document]. URL <https://www.johnsnowlabs.com/how-to-detect-languages-with-python-a-comprehensive-guide/>
- Schaffhauser, S., 2022. Using the common crawl as a data source [WWW Document]. URL <https://medium.com/@samuel.schaffhauser/using-the-common-crawl-as-a-data-source-693a41b3baa9>
- Suen, W., 2018. Large scale graph mining with spark (part 2) [WWW Document]. URL <https://towardsdatascience.com/large-scale-graph-mining-with-spark-part-2-2c3d9ed15bb5>